

WebApp - AtualizaIndicador

Processo independente de coleta e atualização dos indicadores

1. Objetivo

O AtualizaIndicador é um processo Python independente do WebApp. Sua responsabilidade é buscar dados em fontes externas, tratar e validar essas informações e atualizar as tabelas de indicadores no PostgreSQL.

A principal decisão arquitetural é separar a **produção e atualização dos dados** da aplicação Flask que os consome e apresenta ao usuário.

2. Conceito principal

```
AtualizaIndicador = PRODUZ / ATUALIZA os dados
WebApp = CONSOME / EXIBE os dados
PostgreSQL = COMPARTILHA os dados entre ambos
```

3. Visão geral

```
APIs externas
|
v
+-----+
| AtualizaIndicador |
| Python |
+-----+
|
| coleta / trata / valida
|
v
+-----+
| PostgreSQL |
+-----+
|
| consulta
v
+-----+
| WebApp |
| Flask |
+-----+
|
v
Usuário
```

4. Responsabilidades

Componente	Responsabilidade
Scheduler	Define quando cada rotina de atualização deve ser executada.
Coleta	Consulta APIs ou outras fontes externas e recebe os dados brutos.
Tratamento	Converte os dados recebidos para a estrutura utilizada pelo projeto.
Validação	Verifica campos, tipos, datas e valores antes da persistência.
Persistência	Insere ou atualiza os indicadores no PostgreSQL.
WebApp	Consulta os dados já preparados e os apresenta ao usuário.

5. Separação entre AtualizaIndicador e WebApp

O AtualizaIndicador não precisa fazer parte das rotas Flask. Os dois processos compartilham o PostgreSQL, mas possuem responsabilidades diferentes.

```
PostgreSQL
/ \
/ \
AtualizaIndicador WebApp
| |
| escreve | consulta
v v
Indicadores Interface Web
```

Essa separação reduz o acoplamento. Uma alteração na API de um indicador pode ser tratada no AtualizaIndicador sem exigir mudanças nas páginas ou rotas do WebApp.

6. Pipeline de atualização

```
API / Fonte externa
|
v
Requisição
|
v
JSON / dado bruto
|
v
Tratamento Python
|
v
Validação
|
v
Persistência SQL
|
v
PostgreSQL
```

Esse fluxo transforma dados externos, que podem possuir formatos diferentes, em informações padronizadas e prontas para serem consumidas pelo WebApp.

7. Scheduler em Python

O agendamento faz parte do processo AtualizaIndicador e não das rotas do Flask. Dessa forma, a atualização acontece automaticamente sem depender de uma visita ao site ou de uma requisição do usuário.

```
Scheduler
|
+-- horário A -> atualizar dólar
|
+-- horário B -> atualizar IPCA
|
+-- horário C -> atualizar temperatura
|
+-- ... novos indicadores
```

A ideia é manter a automação concentrada no ecossistema Python, deixando o Flask responsável somente pela aplicação web.

8. Indicadores

O desenho permite trabalhar com diferentes indicadores de maneira independente. Entre os dados já trabalhados no projeto estão dólar, IPCA e temperatura.

```
AtualizaIndicador
|
+-- Dólar
|
+-- IPCA
|
+-- Temperatura
|
+-- Novo indicador A
|
+-- Novo indicador B
```

A inclusão de novos indicadores deve seguir o mesmo ciclo: coleta, tratamento, validação e persistência.

9. Relação com a infraestrutura do WebApp

O AtualizaIndicador acessa as fontes externas e o banco diretamente. Ele não precisa passar pelo Apache, Gunicorn ou Flask para atualizar os indicadores.

```
INTERNET
|
+-----+-----+
| |
v v
APIs externas Apache
| |
v Gunicorn
+-----+ |
| AtualizaIndicador | Flask
+-----+-----+ |
| |
+-----> PostgreSQL <-----+
```

10. Independência operacional

Como o AtualizaIndicador é independente da camada web, a coleta pode continuar mesmo que o Apache, Gunicorn ou Flask estejam temporariamente indisponíveis, desde que o processo de atualização, a rede e o PostgreSQL estejam funcionando.

```
AtualizaIndicador
|
+-- não depende do Apache
+-- não depende do Gunicorn
+-- não depende de rota Flask
+-- não depende de acesso do usuário
|
+-- depende das fontes externas
+-- depende do PostgreSQL
```

11. Vantagens da separação

Manutenção: mudanças em APIs ficam concentradas no AtualizaIndicador.

Desacoplamento: o WebApp não precisa saber como os dados foram coletados.

Automação: os indicadores podem ser atualizados mesmo sem usuários acessando o sistema.

Expansão: novos indicadores podem ser adicionados mantendo o mesmo padrão.

Diagnóstico: problemas de coleta podem ser investigados separadamente de problemas da aplicação web.

12. Diagnóstico rápido

Quando um indicador deixar de atualizar, a investigação pode seguir este caminho:

1. O Scheduler executou a rotina?
|
2. A API/fonte externa está disponível?
|
3. A requisição retornou dados?
|
4. O formato da resposta mudou?
|
5. O tratamento Python funcionou?
|
6. Os dados passaram pela validação?
|
7. A conexão com PostgreSQL está ativa?
|
8. O INSERT/UPDATE foi executado?
|
9. O WebApp está consultando o registro correto?

13. Resumo mental

```
AtualizaIndicador
=
Scheduler
+
Coleta
+
Tratamento
+
Validação
+
Persistência

Resultado:
dados prontos no PostgreSQL
para consumo pelo WebApp.
```

Em uma frase: **o AtualizaIndicador mantém os dados prontos; o WebApp apenas os consome e apresenta.**

14. Conclusão

O AtualizaIndicador funciona como um processo de integração e atualização de dados separado da aplicação Flask. Essa divisão mantém as responsabilidades claras: fontes externas fornecem os dados, o AtualizaIndicador coleta e prepara essas informações, o PostgreSQL as persiste e o WebApp as consulta para apresentação. A arquitetura reduz dependências e facilita manutenção, diagnóstico e inclusão de novos indicadores.